

ENGR 102 Summer Bridge

Day 11 Instructor Key

While Loops, Update Order, and Termination Evidence

20 points • approximately 20-25 minutes

Name		UIN		Section	
------	--	-----	--	---------	--

Boolean-operator convention. Python operators appear as [and](#), [or](#), and [not](#). Their lowercase spelling is required in code.

Show intermediate state for trace credit. Program credit requires one coherent solution. Unless a prompt says otherwise, give exact Python 3 output.

Question 1. Trace a while loop that crosses its target.

5 points

```
current = 2
target = 7
steps = 0

while current < target:
    current = current + 2
    steps = steps + 1
    print(current, steps)
```

Give the complete pass ledger and explain why the final value can exceed target.

Answer and explanation

Passes store and print 4 1, 6 2, and 8 3. The condition is checked before each update; 6 is still below 7, so the next +2 update crosses the target. The loop stops only after current becomes 8.

Question 2. Diagnose a non-progressing loop.

4 points

```
value = 3

while value < 10:
    print(value)
    value + 2
```

Explain why the loop does not terminate, give the minimal repair, and list repaired output.

Answer and explanation

value + 2 computes a result but does not store it, so value remains 3 and the condition stays True. Repair with value = value + 2. Repaired output is 3, 5, 7, and 9 on separate lines.

Question 3. Program construction**11 points**

Write one complete Python program that satisfies every requirement.

- Read start and target as integers. Valid floors are 1 through 10 inclusive.
- Print invalid if either floor is outside the interval and do not run a loop.
- Print already there if the two valid floors are equal.
- Otherwise use a while loop to move exactly one floor per pass toward target and print each newly reached floor.
- After movement, print Steps: followed by the number of loop passes.

Answer and explanation

```
current = int(input())
target = int(input())

if current < 1 or current > 10 or target < 1 or target > 10:
    print("invalid")
elif current == target:
    print("already there")
else:
    steps = 0
    while current != target:
        if current < target:
            current = current + 1
        else:
            current = current - 1
        steps = steps + 1
        print(current)
    print(f"Steps: {steps}")
```

Validation and equality are handled before repetition. The loop condition means movement is unfinished, and exactly one directional branch changes current by one each pass.

Inputs 2 and 5 print 3, 4, 5, then Steps: 3. Inputs 5 and 2 print 4, 3, 2, then Steps: 3. This protects both update directions.

Scoring guidance

2 validation/equality, 2 correct while condition, 3 directional update/progress, 2 output placement/count, 2 bidirectional tests.